# Informed Search

# Informed Search

- Search efficiency would improve greatly if there is a way to order the choices so that the most promising are explored first.
  - This requires domain knowledge of the problem.
- Add domain specific knowledge to select what is the best one to continue searching along the path
  - Define a heuristic function, h(n) that estimates the **goodness** of a node n, based on domain specific information that is computable from the current state description.
- Heuristics is knowledge about the domain that helps to undertake focused search
  - It is rule of thumb, or guess that limits the search space towards the goal

Example: 8- puzzle

# **Heuristic function,** h(n)

- It is an estimate of how close we are to a goal state
- Heuristic function h(**n)**

  **h(n)** = estimated cost of path from state n to a goal state.

- What good heuristic function for the following problems:
  - **Route finding:** straight line distance
  - **8-puzzle:** (i) Number of mismatched tiles and (ii) Manhattan or city block distance (sum of distances each tile is from its goal position)
  - **8-queens:** Min-conflict or max conflict – (that is, the number of attacking queens)
- Note that:
  - $h(n) \geq 0$ for all nodes n
  - $h(n) = 0$ if n is a goal node
  - $h(n) = \infty$ if n is a dead-end from which a goal can't be reached

# Best first search

- It is a generic name to the class of informed methods
- Nodes are ordered in the open list so that the one with the best evaluation (minimum cost) is expanded first
  - Evaluation function, **f** incorporates domain specific information and determines the desirability of expanding the node
  - It aims to find low-cost solutions towards goal node
- There are two measures
  - The use of the path cost **g** to decide which path to extend further (e. g. uniform cost search).
  - The use of some estimate of the cost of the path from current state to the goal state (heuristics **h**)
  - The first measure may not direct search towards the goal. It needs to be supplemented by the second measure

# Best First Search Approaches

- Two approaches to find the shortest path:
  - **Greedy search**: minimizes estimated cost to reach a goal
  - **A*-search**: minimizes the total path cost
- When expanding a node *n* in the search tree, greedy search uses the estimated cost to get from the current state to the goal state, define as *h(n)*.
  - In route finding problem *h(n)* is the straight-line distance
- We also possess the sum of the cost to reach that node from the start state, define as *g(n)*.
  - In route finding problem this is the sum of the step costs for the search path.
- For each node in the search tree, an evaluation function *f(n)* can be defined as the sum of these functions.

$$f(n) = g(n) + h(n)$$

# Algorithm for Best First search

- **Implementation**:
  - Sort states (in ascending order) by the heuristic estimate to the goal
  - Expand first the one with the minimum cost for which the estimate is minimized

```
function Best_first_search (problem, eval_fun){
    open = (C_0); //put initial state C_0 in the List
    closed = {}; /maintain list of nodes examined earlier
    while (not (empty (open))) {
        f = remove_first(open);
        if IsGoal (f) then return (f); //goal test
        closed = append (closed, f);
        l = not-in-set (Successors (f), closed );
        open = merge ( rest(open), l, eval_fun); //sort the list using
                    evaluation function
    }
    return ('fail')
}
```

# Admissibility

- Search algorithms (such as greedy and A*- search) are admissible when the heuristic function never over estimated the actual cost so that
  - the algorithm always terminates in an optimal path from the initial state to goal node if one exists.
- check whether estimated cost h(n) is not overestimated
  - h*(n): Actual cost of shortest path from n to z (not known)
  - h(n) is said to be an admissible heuristic function if for all n, $h(n) \leq h^*(n)$
  - The closer h to h* the fewer extra nodes that will be expanded
  - Using an admissible heuristics guarantees that the solution found by searching algorithm is optimal

# Consistency of a Heuristic Function

- A heuristic function is said to be consistent, or monotone
  - If its estimate is always less than or equal to the estimated distance from any neighboring vertex to the goal, plus the cost of reaching the neighbor.
  - Formally, for every node **N** and each successor **P of N**, the estimated cost of reaching the goal from **N** is no greater than the step cost of getting to **P** plus the estimated cost of reaching the goal from P.
    - i.e. **$h(N) \leq c(N,P) + h(P)$ and $h(G)=0$**
      - Where
        - » H is the consistent heuristic function
        - » N is any node in the graph
        - » P is any descendent of N
        - » G is any goal node
        - » c(N,P) is the cost of reaching node P from N

# Consistency of a Heuristic Function (Cont.)

- $h(goal)=0$ AND
- $h(s) \leq c(s,n) + h(n)$ for all $s \in S$ and
  - all $n \in$ neighbor(s),
  - $c(s,n)$=step cost from s to n
  - neighbors(s)= set of all states one step from s.
- A consistent heuristic is also admissible
  - The converse, however, is not always true. (e.g. should be assignment)

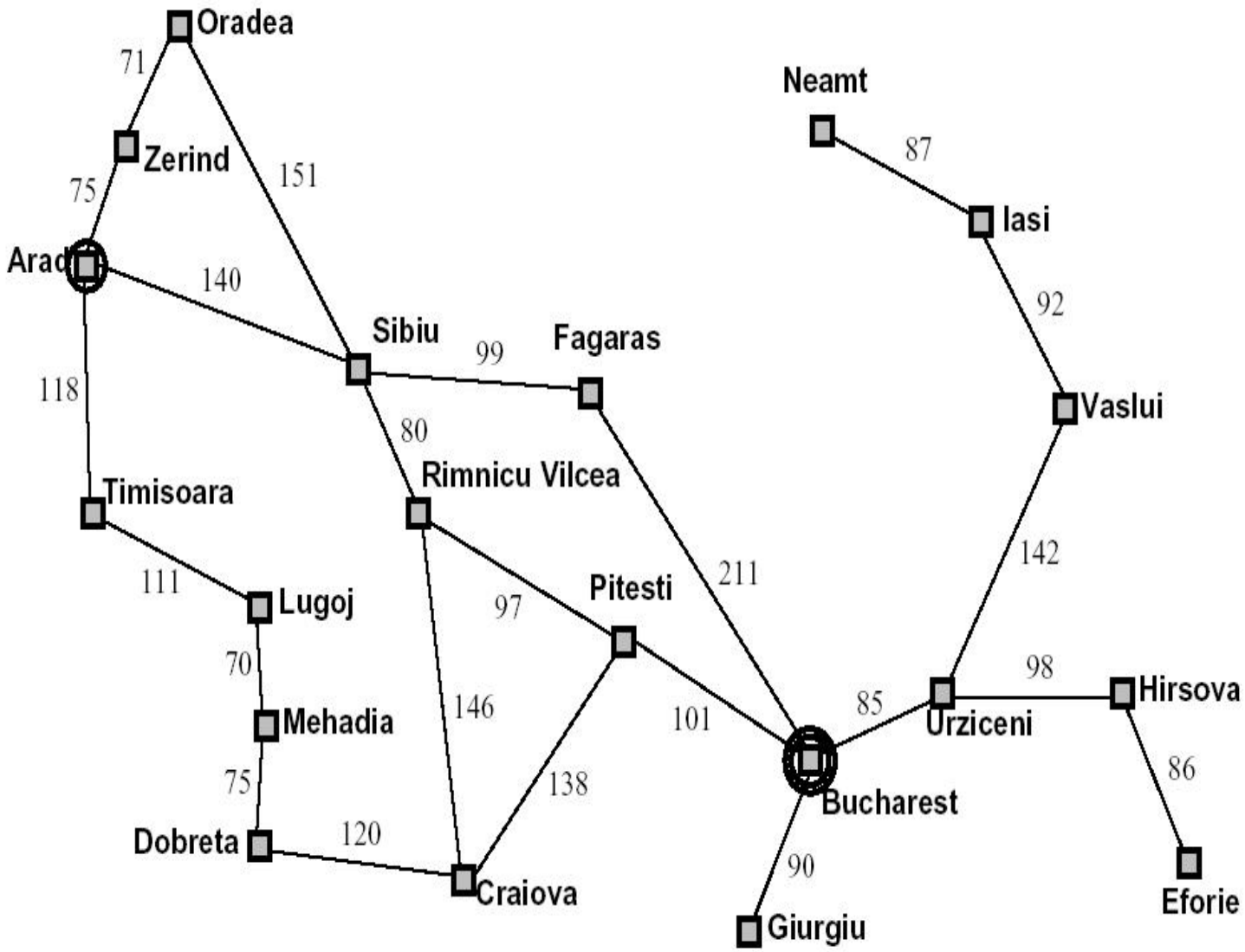
# Greedy search

- A best first search that uses a heuristic function h(n) alone to guide the search
  - Selects node to expand that is closest (hence it's greedy) to a goal node
  - The algorithm doesn't take minimum path costs from initial to current nodes into account, it just go ahead optimistically and never looks back.
- **Implementation**: expand 1st the node closest to the goal state, i.e. with evaluation function f(n) = h(n)
  - h(n) = 0 if node n is the goal state
  - Otherwise h(n) ≥ 0; an estimated cost of the cheapest path from the state at node n to a goal state

**Example 1**: Route finding problem

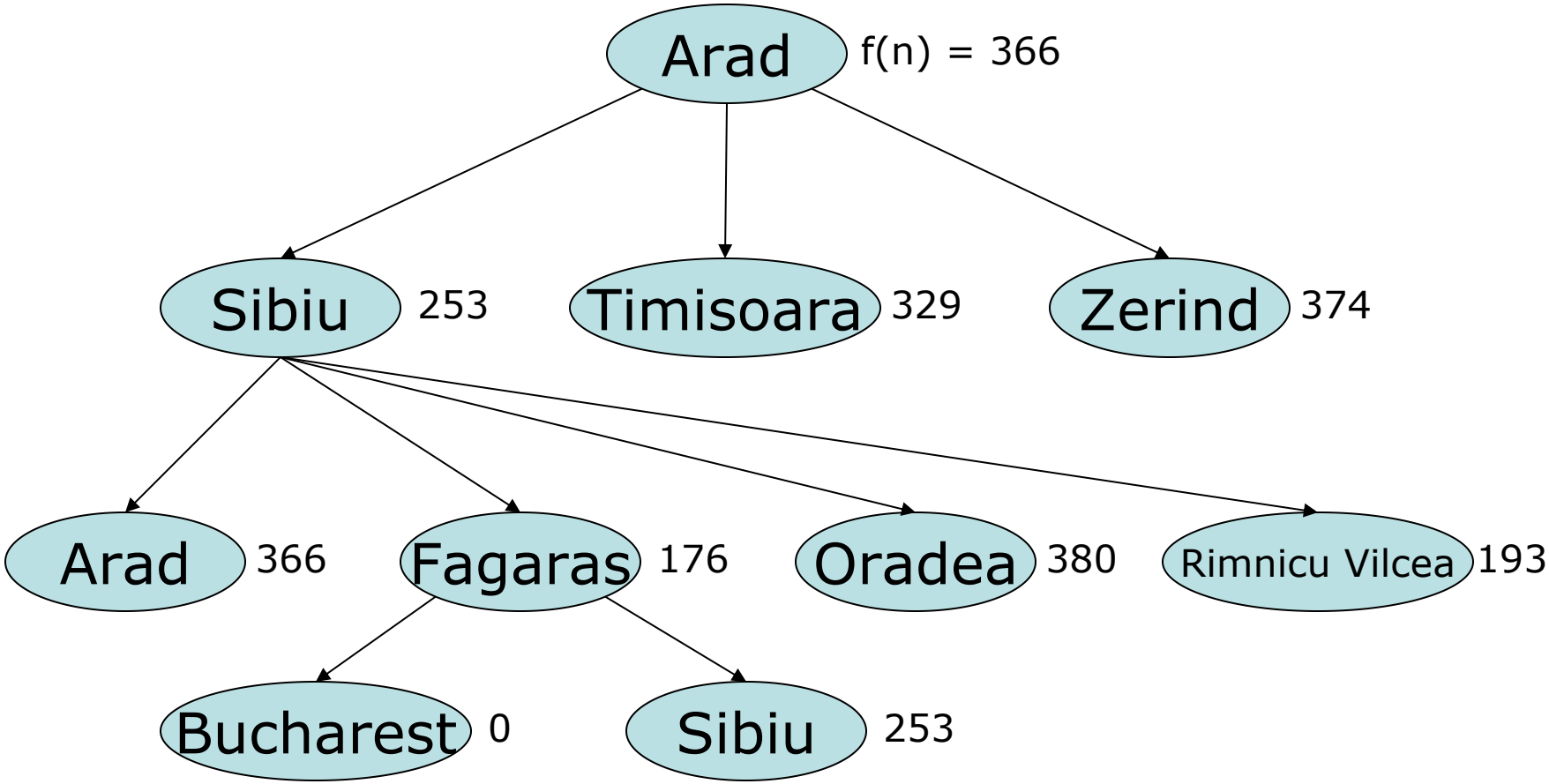
**Example 2**: Can we arrange the search in **the 8-puzzle** so that a move will minimize the distance b/n the initial state & the goal arrangement

# Exercise




| Straight−line distance to Bucharest | |
|---|---|
| Arad | 366 |
| Bucharest | 0 |
| Craiova | 160 |
| Dobreta | 242 |
| Eforie | 161 |
| Fagaras | 178 |
| Giurgiu | 77 |
| Hirsova | 151 |
| Iasi | 226 |
| Lugoj | 244 |
| Mehadia | 241 |
| Neamt | 234 |
| Oradea | 380 |
| Pitesti | 98 |
| Rimnicu Vilcea | 193 |
| Sibiu | 253 |
| Timisoara | 329 |
| Urziceni | 80 |
| Vaslui | 199 |
| Zerind | 374 |

# Solution to Route Finding




Is Arad → Sibiu → Fagaras → Bucharest optimal?
Total cost = 450

# Properties

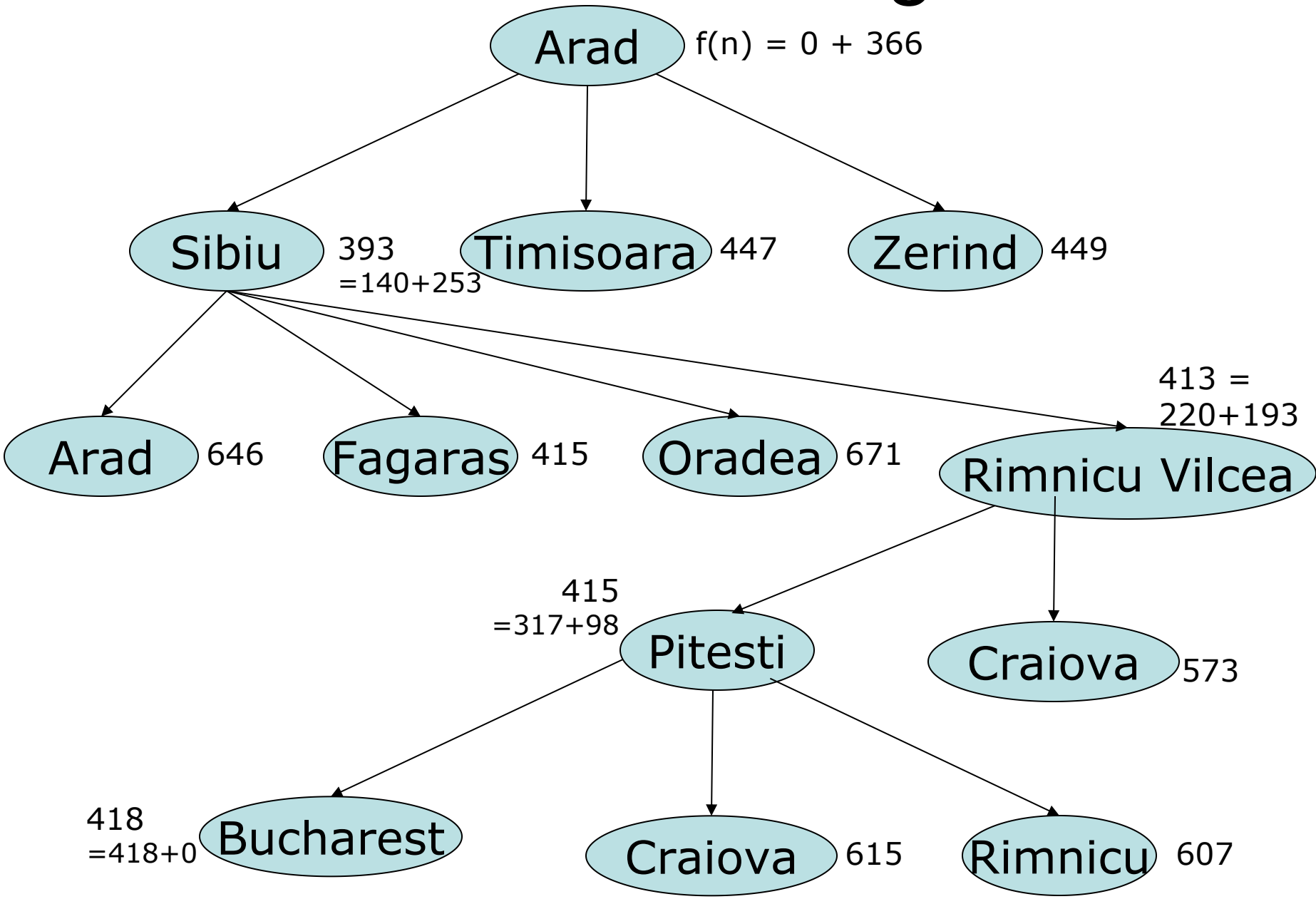
- **Incomplete**:
  - may not reach goal state because it may start down an infinite path and never return to try other possibilities
- **Not optimal**
  - if estimated heuristic cost is not admissible
- **Takes memory**
  - It retains all the nodes in memory
  - Space complexity is exponential: $O(b^m)$, where m is the maximum depth of the search space and b is branching factor
- **Time complexity**
  - It takes exponential time: $O(b^m)$
- With good heuristic function the space and time complexity can be reduced substantially
- The problem with greedy search is that it didn't take costs so far into account.
  - We need a search algorithm (like A* search) that takes into consideration this cost so that it avoids expanding paths that are already expensive

# A*-search Algorithm

- It considers both estimated cost of getting from n to the goal node **h(n)**, and cost of getting from initial node to node n, **g(n)**
- Apply three functions over every nodes
  - g(n): Cost of path found so far from initial state to n
  - h(n): Estimated cost of shortest path from n to z
  - f(n): Estimated total cost of shortest path from a to z via n
  - Evaluation function f(n) = h(n) + g(n)
- **Implementation**: Expand the node for which the evaluation function f(n) is lowest
  - Rank nodes by f(n) that goes from the start node to goal node via given node

Example: Route finding problem

# Solution to Route Finding



# Properties

- A* search is complete, optimal, and optimally efficient for any given admissible heuristic function
- **Complete**
  - It guarantees to reach to goal state
- **Optimality**
  - If the given heuristic function is admissible, then A* search is optimal.
  - **Proof** optimality of A* search
- **Takes memory:**
  - It keeps all generated nodes in memory
  - Both time and space complexity is exponential
- Time and space complexity of heuristic algorithms depends on the quality of the heuristic function.
  - Worst case is exponential

# Local Search

- In many problems, the path to the goal is irrelevant; the goal state itself is the solution
- Being able to ignore the path offers two main advantages:
  - Less memory is used, generally a constant amount.
  - A reasonable solution can be found in infinite search spaces for which systematic search would be unsuited.
  - They are also very useful for optimization problems where the goal is to find the best state according to a function. Many of these problems are not suitable for standard search.

**Example**: n-queens problem: Put n queens on an n × n board with no two queens on the same row, column, or diagonal.

  - In these cases, we can use local search algorithms that keep a single current state, try to improve it.

- The general idea of local search is to start with a complete configuration and to make modifications to improve its quality
- **Implementation**:
  - Consider all states laid out on the surface of a landscape
  - It moves around the landscape trying to find the optimal solutions
- Search algorithms: Hill climbing, Simulated annealing, ….

# Hill–climbing Algorithm

- The idea here is that we don't keep track of our path, we don't check for repeated states;
  - It just keep track of the current state
  - it looks around current position & moves **uphill** or **downhill**.
- Hill climbing is best suited to problems where the heuristic improves the closer it gets to the solution;
  - it works poorly where there are sharp drop-offs. It assumes that local improvement will lead to global improvement.
- **Implementation**:
  - Evaluate the successors and chose the best state that leads us closer to the goal (according to the heuristic estimate), and continue from there

# Hill climbing algorithm

```
Function hill_climbing(problem) returns a solution state
  variable: current and next (nodes)
  current = initial state
  while current ≠ NULL do
    next = a highest value successor of current
    if (value (next) < value (current)) then
        return current
     current = next
  end loop
end function
```

# Properties

- If an accurate heuristic measure is available in the domain, and if there are no local maxima, the algorithm is
  - fast to reach to goal state
    - it just looks a head to determine if any successor is better than the current state
  - takes less memory
    - space requirement is constant since current state is the only node stored
- Incomplete and not optimal
  - Since it didn't remember previously generated nodes, it may get back to the node that is already examined and hence become infinite
- No special implementation data structure required since hill climbing *discards old* nodes
  - In case the path followed doesn't lead to goal state, there is no way to backtrack or to jump to an alternate path, since no information is remembered.

# Hill – climbing…

Example: **The 8-puzzle problem**

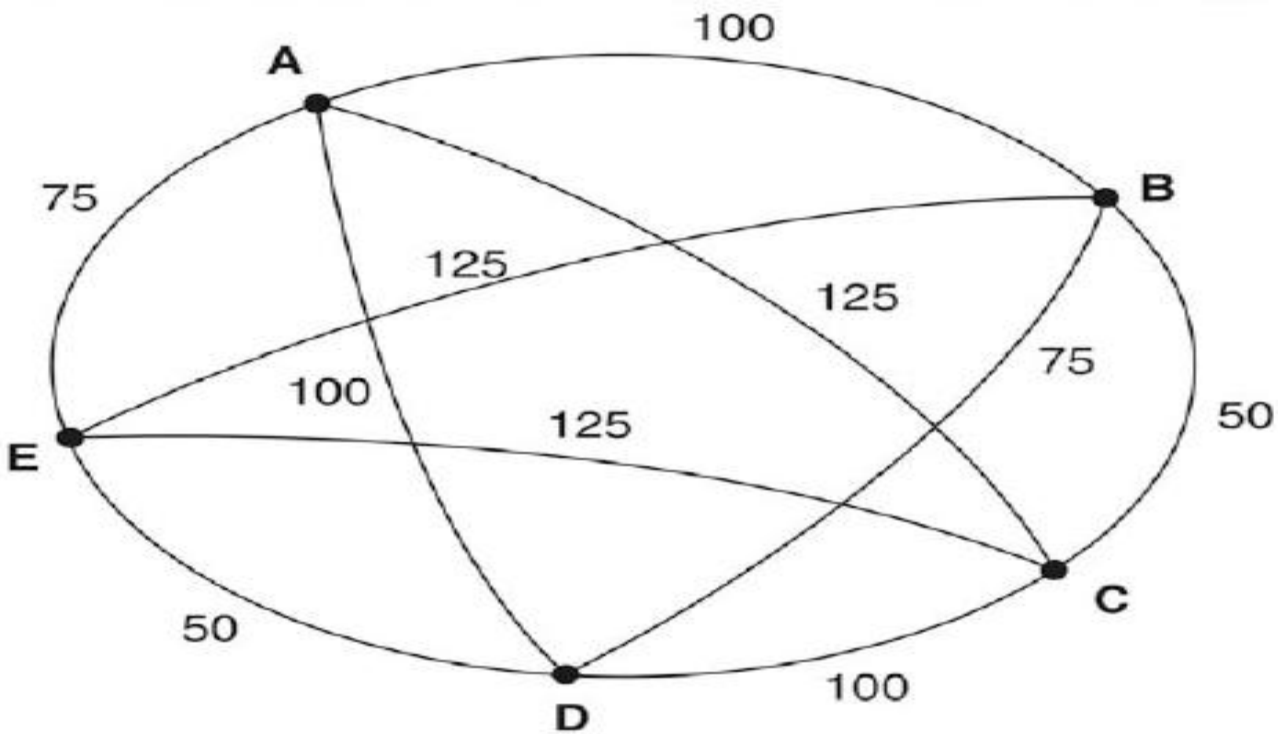
Rearrange the titles until the final goal is achieved.
For evaluation consider the number of mismatched tiles.

| | | |
|---|---|---|
| 2 | 8 | 3 |
| 1 | 6 | 4 |
| 7 | | 5 |

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 8 | | 4 |
| 7 | 6 | 5 |

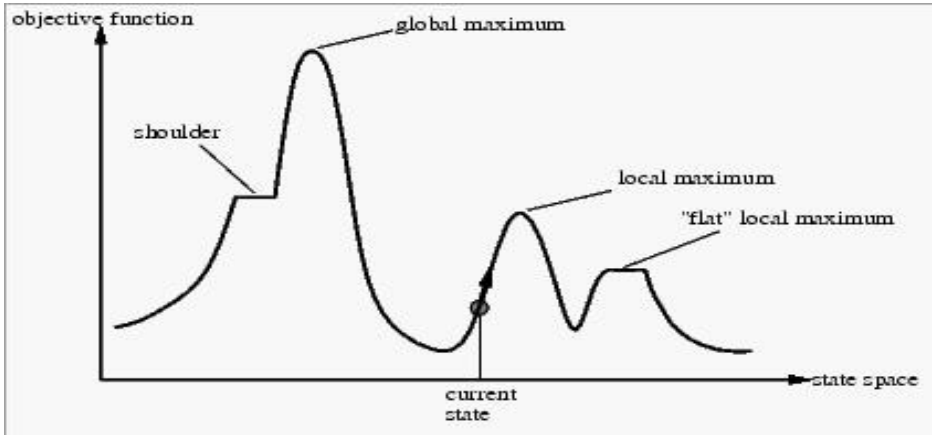
# Hill – climbing…

- **Traveling Sales person problem:** Given a set of cities, with costs of getting from one to the other, find a minimum cost route that goes through each of them exactly once.
  - Consider the following example: Salesperson lives in city **A.** He has to visit all the 5 cities, A to E, and must return home afterwards. Goal is circle from **A** to **A** with minimal cost. Find shortest path for travel minimize cost and/or time of travel



# Problems with hill climbing

- **Local Maximum:** Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.
- **Global Maximum:** Global maximum is the best possible state of state space landscape. It has the highest value of objective function.
- **Current state:** It is a state in a landscape diagram where an agent is currently present.
- **Flat local maximum:** It is a flat space in the landscape where all the neighbor states of current states have the same value.
- **Shoulder:** It is a plateau region which has an uphill edge.
- May lead to **local maxima** – once it reaches to local maxima, it can't go to global max. It caught by small foothills.
- Walking in the **Plateau** - situations where all of the local steps are indistinguishable; all of them have the same value.
- **ridges** traversal - there is a direction we would like to move, but none of the allowed steps actually takes us in that direction.

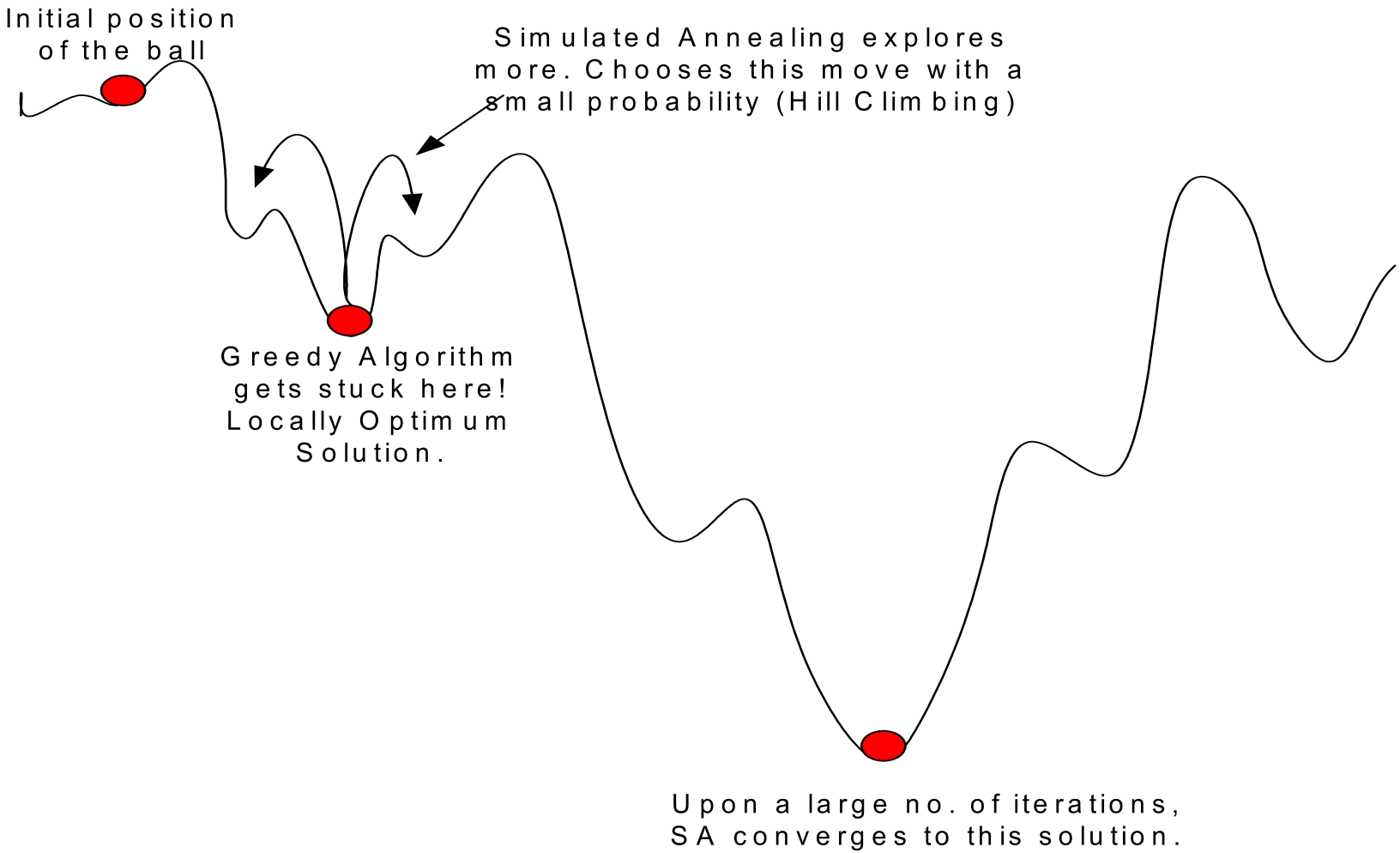


# Possible Improvements

- Random-restart (Simulated Annealing)
  - To avoid the problems of hill-climbing, we need to mix some randomness with the greedy uphill steps such that
    - incase the algorithm stuck on a local maximum, we could allow the search to take some down hill steps to escape the local maximum
- The basic idea is that, instead of always going up the sharpest incline, pick a random direction.
  - If the move improves the situation; i.e. if it goes uphill, go there and execute it;
  - otherwise, go downhill with probability which depends on how bad the move is (the probability decreases exponentially with the badness of the move.

# Simulated Annealing vs. Hill climbing

Ball on terrain example: The ball is initially placed at a random position on the terrain. From the current position, the ball should be fired such that it can only move one step left or right. What algorithm should we follow for the ball to finally settle at the lowest point on the terrain?



# Heuristic Search Methods

| **Name** | **Time Comp.** | **Space Comp.** | **Optimal** | **Complete** |
|---|---|---|---|---|
| Greedy | $O(b^m)$ | $O(b^m)$ | No | No |
| A* | $O(b^d)$ | $O(b^d)$ | yes optimal if *h'(n)* never overestimates | yes |
| Hill Climbing | $b^m$ | O(1) | No | No, it needs a way of escaping from local minima |
| Simulated Annealing | unbounded | O(1) | Yes | will find optimal solution if cooling is slow enough and as time goes to infinity |

*b* is the branching factor; *d* is the depth of solution; *m* is the maximum depth of the search tree; *N* the number of nodes

# Constraint satisfaction problem (CSP)

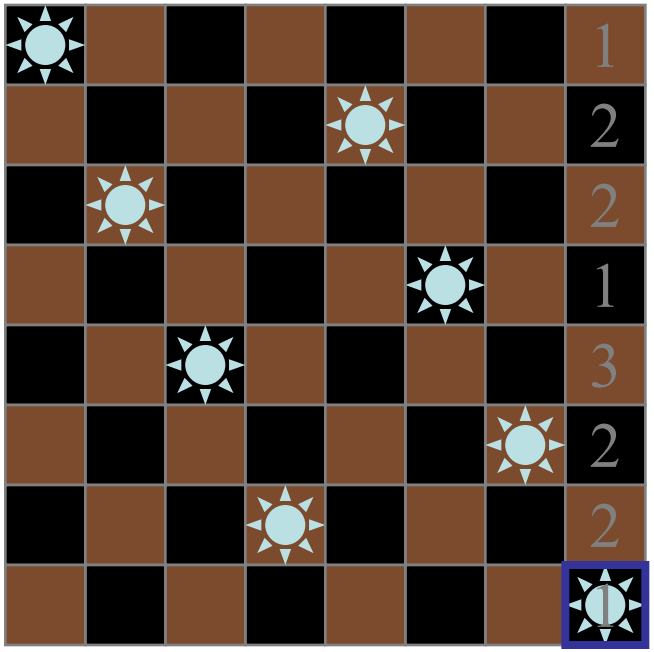
- It can be solved by iterative improvement method
  - First assign values to all variables
  - Then apply modifications operators to move the configuration towards the goal
- Modification operators simply assign a different value to a variable
  - In choosing a new value for a variable, the most obvious heuristic is to select the value that results in the minimum number of conflicts with other variables - the **min-conflicts heuristic or max-conflicts heuristic**
- Algorithms that solves CSPs problems in this fashion are called **heuristic repair methods**

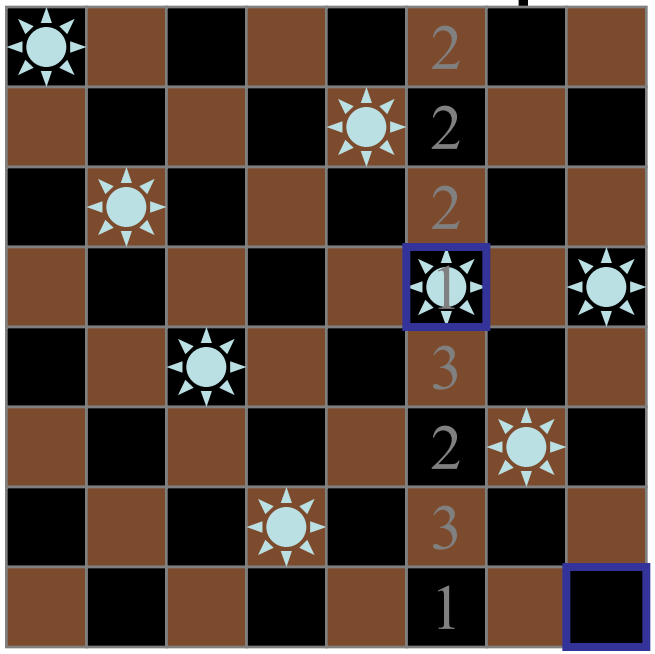
Example: the 8-queens problem

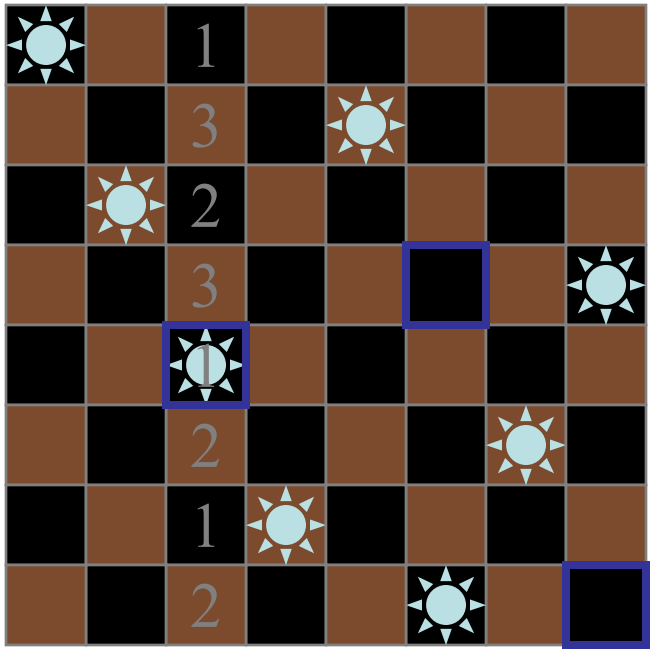
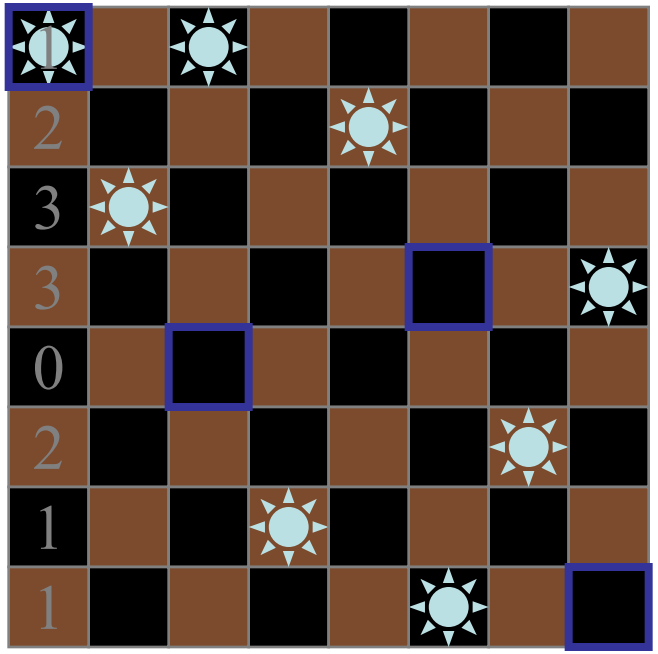
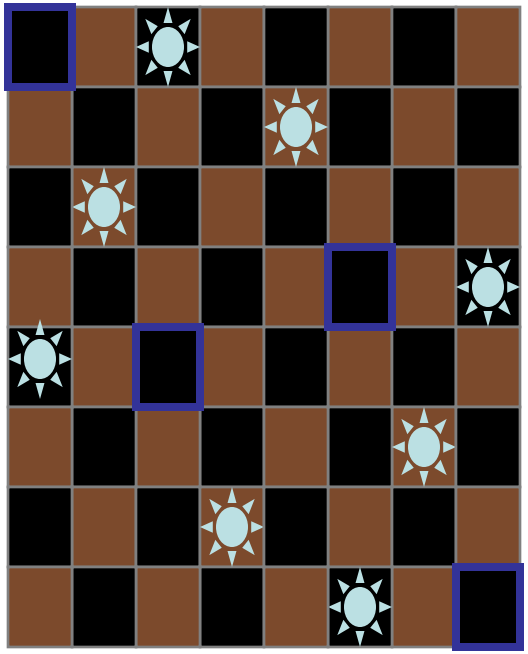
# The 8-queens problem formulation

- Place 8 queens on a board so that no queen is attacking another queen (see next slide for detail).
- Initial State:
  - 8 queens placed randomly on the board, one per column.
- Successor function:
  - States that obtained by moving one queen to a new location in its column.
- Heuristic/objective function: The number of pairs of attacking queens.
  - Use **min-conflict** heuristic to move attacked queen column-wise.
  - Use **max-conflict** to move the most attacked queen
  - Break any ties randomly.
- Cost function: each move costs 1.

# 8-Queens Example 1




Solution found in 4 steps using min-conflicts heuristic